

1. Syntax-Directed Definitions (SDD)

A **Syntax-Directed Definition** is a way to associate **semantic rules (meaning)** with the **grammar of a language**.

Key Idea:

- Each grammar symbol has **attributes**.
- Each production has **semantic rules** to compute attribute values.

Example:

For expression:

$$E \rightarrow E1 + T$$

Rule:

$$E.val = E1.val + T.val$$

Types of Attributes:

- **Synthesized Attributes** $\rightarrow$ Computed from child nodes (bottom-up)
 - **Inherited Attributes** $\rightarrow$ Passed from parent/siblings (top-down)
-

2. Dependency Graphs

A **Dependency Graph** shows how attributes depend on each other.

Features:

- Nodes $\rightarrow$ Attributes
- Edges $\rightarrow$ Dependency between attributes
- Used to determine **evaluation order**

Important:

- If graph has **no cycles** $\rightarrow$ **valid evaluation**
 - If graph has **cycle** $\rightarrow$ **cannot evaluate**
-

3. S-Attributed Definitions

A special type of SDD where:

- Only **synthesized attributes** are used

Properties:

- Easy to implement using **bottom-up parsing (LR parser)**
- Evaluation happens **from leaves to root**

Example:
$$E \rightarrow E1 + T$$
$$E.val = E1.val + T.val$$

4. L-Attributed Definitions

An SDD where:

- Allows **both synthesized and inherited attributes**
- But with restriction:
 - Inherited attributes depend only on:
 - Parent
 - Left siblings

Properties:

- Can be evaluated in **top-down parsing (LL parser)**
 - Very useful in **real compilers**
-

5. Application of Syntax-Directed Translation (SDT)

Used in compilers for:

- **Type checking**
- **Intermediate code generation**
- **Symbol table management**
- **Expression evaluation**

Example:

Converting infix $\rightarrow$ postfix:

$$E \rightarrow E1 + T \{ \text{print}('+') \}$$

6. Syntax Directed Translation Schemes (SDTS)

A **translation scheme** is a grammar with **embedded program code (actions)**.

Key Difference from SDD:

- SDD $\rightarrow$ Only rules
- SDTS $\rightarrow$ Includes **actions inside grammar**

Example:

$E \rightarrow E1 + T \{ \text{print('+')} \}$

Placement:

- Actions can be placed:
 - At beginning
 - Middle
 - End of production
-

7. Intermediate Code Generation

Intermediate code is a **machine-independent code** generated between:

- Source code $\rightarrow$ Machine code

Advantages:

- Easy optimization
 - Machine independent
 - Simplifies compiler design
-

8. Variants of Syntax Trees

(a) Parse Tree

- Shows full grammar structure
- Contains all symbols (terminals + non-terminals)

(b) Abstract Syntax Tree (AST)

- Simplified version
- Removes unnecessary grammar details

Example:

Expression:

$a + b * c$

AST:

```
  +
 /\
a  *
 /\
b  c
```

9. Three Address Code (TAC)

A common intermediate code format.

Form:

$x = y \text{ op } z$

Types:

1. Binary operation

$t1 = a + b$

2. Unary operation

$t2 = -a$

3. Assignment

$x = y$

4. Jump

if $x < y$ goto L1

Example:

Expression:

$a + b * c$

TAC:

$t1 = b * c$

$t2 = a + t1$

10. Control Flow

Control flow shows the **order in which statements execute**.

Control Flow Graph (CFG):

- Nodes $\rightarrow$ Basic blocks
- Edges $\rightarrow$ Flow of control

Basic Block:

A sequence of statements with:

- Single entry
- Single exit

Example:

```

if (a < b)
    x = y + z;
else
    x = y - z;

```

Flow:

- Condition → True block / False block → Merge

BASIC BLOCKS

Fact(n)

```

{
  int f = 1;
  for (i = 2; i ≤ n; i++)
    f = f * i;
  return f;
}

```

IC

TAC

```

1) f = 1
2) i = 2
3) if (i > n) goto 8
4) t1 = f * i
5) f = t1
6) i = i + 1
7) goto 3
8) goto calling f^n

```

B₁

B₂

B₃

B₄

→ In order to find basic blocks, we need to find leaders in the three address code.

→ A basic block will start from one leader to the next leader but not including next leader.

Finding Leaders in a Basic Block -

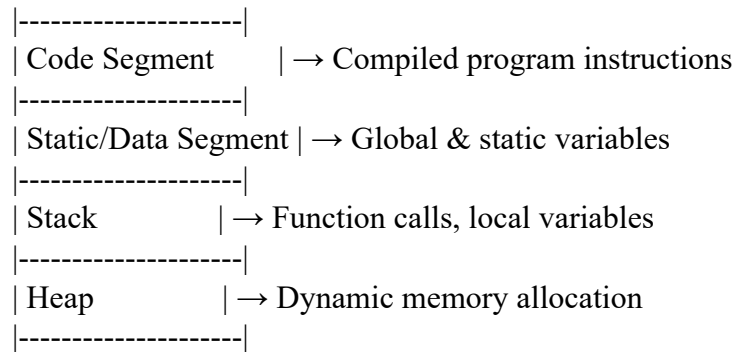
1. The first three address instruction in the Intermediate Code is a leader.
2. Any instruction that is the target of Conditional or unconditional jump is a leader.
3. Any instruction that immediately follows a Conditional or unconditional jump is a leader.

Runtime Environment and Code Generation

◆ 1. Storage Organization

A program's memory is divided into different regions, each serving a specific purpose.

✦ Memory Layout



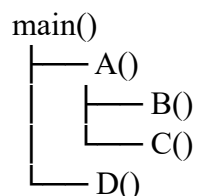
◆ Key Points:

- **Stack grows downward**, heap grows upward
 - Stack is **automatic**, heap is **manual/controlled**
 - Efficient management is crucial for performance
-

◆ 2. Activation Trees

An **activation tree** represents how functions are called during execution.

✦ Example:



◆ Explanation:

- Each node = function call
 - Parent → calls child
 - Shows **nested structure of execution**
 - Used to understand recursion and control flow
-

◆ 3. Activation Records (Stack Frames)

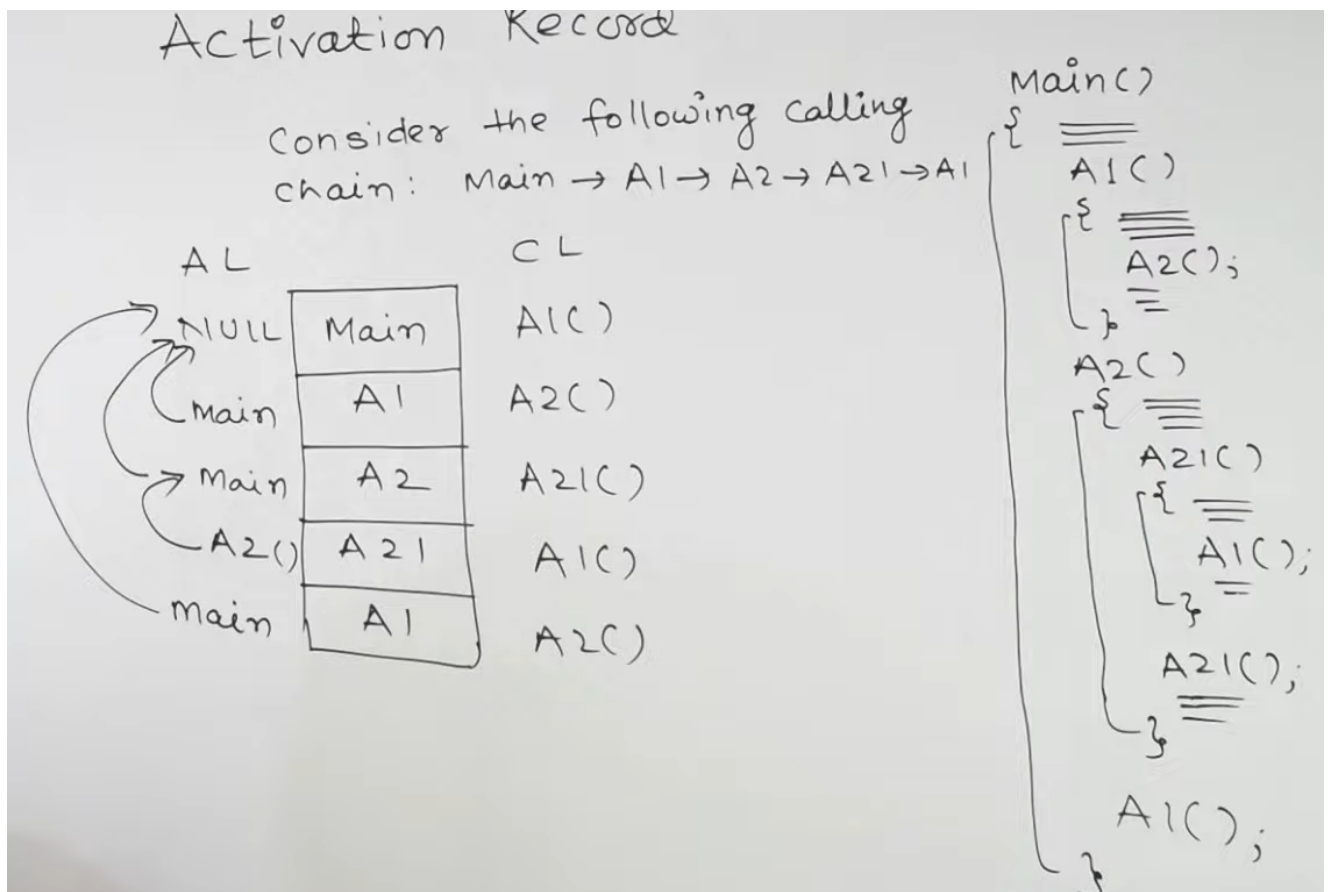
Each function call creates an **activation record** stored in the stack.

✦ Structure:

Actual Parameters
Return Value
Return Address
Control Link
Access Link
Local Variables
Temporary Variables

◆ **Fields:**

- **Return Address** → where control goes back
- **Control Link** → pointer to caller's record
- **Access Link** → used for nested scopes
- **Locals/Temps** → store variables



◆ **4. Calling Sequence**

Steps executed during a function call:

◆ **Caller Actions:**

1. Evaluate and push arguments
2. Save registers (if needed)
3. Transfer control to function

◆ **Callee Actions:**

4. Create activation record
5. Initialize local variables
6. Execute function body

◆ **Return Phase:**

7. Store result
8. Restore registers
9. Return control to caller

◆ **5. Heap Management**

Heap is used for **dynamic allocation of memory at runtime**.

◆ **Operations:**

- malloc() / new → allocate memory
- free() / delete → release memory

◆ **Problems:**

- **Memory Leak** → memory not freed
- **Fragmentation** → unused gaps in memory
- **Dangling Pointer** → pointer to freed memory

◆ **6. Garbage Collection (Introduction)**

Automatic technique to reclaim unused memory.

◆ **Methods:**

1. **Reference Counting**
 - Each object keeps count of references
 - When count = 0 → delete

2. Mark and Sweep

- Mark reachable objects
- Sweep unmarked ones

◆ Advantage:

- Reduces programmer effort
 - Avoids memory leaks
-

◆ 7. Code Generation

Final phase of compiler:

- Converts **intermediate code** → **target machine code**

◆ Goals:

- Correctness
 - Efficiency
 - Optimal use of registers
-

◆ 8. Issues in Code Generator

◆ Important Issues:

- **Instruction Selection** → choose best machine instruction
 - **Register Allocation** → limited registers must be used efficiently
 - **Evaluation Order** → affects performance
 - **Memory Access** → minimize slow memory operations
-

◆ 9. The Target Language

The output language of compiler:

- Machine code or Assembly

◆ Considerations:

- Instruction set
- Number of registers
- Addressing modes
- CPU architecture

◆ 10. Basic Blocks

A **basic block** is a straight-line code sequence.

✦ Properties:

- No branching inside
- Only one entry and one exit

Example:

```
t1 = a + b
t2 = t1 * c
t3 = t2 - d
```

◆ 11. Flow Graphs

A **flow graph** represents control flow using basic blocks.

◆ Components:

- Nodes $\rightarrow$ Basic blocks
- Edges $\rightarrow$ Possible flow of control

Use:

- Helps in optimization
- Identifies loops and branches

◆ 12. Optimization of Basic Blocks

Improves efficiency within a block.

◆ Techniques:

1. Constant Folding

$a = 2 + 3 \rightarrow a = 5$

2. Common Subexpression Elimination

$t1 = a + b$
 $t2 = a + b \rightarrow \text{reuse } t1$

3. Dead Code Elimination

$x = 5$ (never used) $\rightarrow$ remove

* Constant Folding:

Replacing an expression that can be computed at compile time by its value.

$$\stackrel{\text{ex}}{=} C = 2 * 3.14 * \pi \Rightarrow C = 6.28 * \pi$$

$$\stackrel{\text{ex}}{=} 2 + 3 + B + C = 5 + B + C$$

* Redundancy Elimination:

$$\begin{array}{l} \swarrow \\ a = b + c \\ \hline e = d + b + c \end{array} \Rightarrow \begin{array}{l} a = b + c \\ e = d + a \end{array}$$

* Strength Reduction:

Replacing a expensive operator by cheaper one.

Expensive

Cheaper

$$x^2 \Rightarrow x * x$$

$$2 * x \Rightarrow x + x$$

$$x/2 \Rightarrow x * 0.5$$

$$B = A * 2 \Rightarrow B = A \ll 1$$

* Algebraic Simplification:

$$x + 0 = 0 + x = x, \quad x - 0 = x$$

$$x * 1 = 1 * x = x, \quad x / 1 = x$$

◆ 13. A Simple Code Generator

Converts **three-address code (TAC)** into machine instructions.

Example:

$$t1 = a + b$$

$$t2 = t1 * c$$

Generated Code:

MOV R1, a
ADD R1, b
MUL R1, c

◆ 14. Peephole Optimization

Optimizes small sets of instructions (local optimization).

◆ Techniques:

- Remove redundant instructions
- Replace costly instructions with efficient ones

Example:

MOV R1, a
MOV a, R1

Optimized:

MOV R1, a

a) Redundant LOAD/STORE elimination

$a = b + c$

~~$R_0 = 5$~~

$a = b + c$

$d = a + e$

b)

LOAD R_0, b

ADD R_0, c

STORE a, R_0

$R_0 = b + c$

1) LOAD R_0, b

2) ADD R_0, c

3) STORE a, R_0

4) LOAD R_0, a

5) ADD R_0, e

6) STORE d, R_0

$a = b + c$

R_0

c) Use of M/C idioms

$i = i + 1$

LOAD R_0, i

ADD $R_0, \#1$

STORE i, R_0

INC i

b) Flow of Control Optimization

1) Avoid Jumps on Jumps

L₁: jump ~~L₂~~ L₄

≡
≡
≡
≡

L₂: jump L₃

≡
≡
≡

L₃: jump L₄

≡
≡

L₄: x = a + b * c

2) Eliminate Dead Code

(0 == 1) int i = 0;

if (i == 1)

{

printf("Dead code");

}